

Testing and Comparing SupHopf Model Implementations in TVB and Neurolib

This notebook/test suite focuses on testing the handling of connectivity and delays in the normal form of a supercritical Hopf bifurcation in Cartesian coordinates^[1].

The first step is identifying the relevant model parameters, and setting them to the same values. For TVB and Neurolib, there are only 2 parameters for both models, `a` which appears in both models, and `omega` in TVB which maps to `w` in neurolib.

There are other parameters relevant for simulation equivalence testing, such as `Dt`, conduction speed, initial parameters etc. For the reason of reproducibility we recommend writing a `Config` class, and a wrapper for each model, so that inputs and outputs match for each model. These can be viewed in files `config.py`, `tvb_model.py` and `neurolib_model.py`.

For parameters relevant to the computation that only appear in one model but not the other, choose values that allow consistent mapping between the models. (Good candidates include powers of ten for scaling, zero for additive terms, or identical "magic constants".)

[1] Paraphrased from TVB documentation of SupHopf model.

Testing of connectivity

The idea behind testing connectivity handling of a simulator is relatively straightforward. Suppose that there are 2 equivalent models written in C, except one of them uses 32bit floats to represent their floating point numbers, and the other uses 64bit doubles. Given enough steps, these models, although equivalent otherwise, will diverge because of rounding errors, giving us a false negative on the models being equivalent.

To avoid this issue, we've come up with the solution of running only 1 step at a time, as if those results are close enough, we know the reasoning behind the models is equivalent.

TODO FINISH

Defining Test Cases

If you've tested `dfun`, test only parameters that don't go in `dfun` directly

Next, we define our parameter sets. There are two approaches: 1. **Target key regions of the bifurcation diagram** 2. **Use brute force sampling across the parameter space, one parameter at a time**

The approach recommended in dfun testing (viz last tutorial) is to use both. However, if you've sufficiently tested the dfun of both models, they can be considered equivalent. If we then make the reasonable assumption that the dfun parameters don't...

There is a general problem with testing of these functions, which is that the regions that are tested need to be vetted, so that we're not looking in the wrong place. Option #1 circumvents this issue by having few enough test cases that manual checking is possible. Option #2 simply ignores this, in favor of having lots of test cases.

As for the one parameter at a time part of option two, this comes from the pragmatic view of the problem as simply a parametrized $f(x)$. If there is a difference in $f(x)$ and $f'(x)$, changing multiple parameters at the same time (and with sufficient sampling density) should reveal this difference just the same as changing only one.

```
In [4]: from config import Config
c = Config(random_seed=42)
print(f"dt = {c.dt}")
print(f"speed = {c.speed}")
print(f"history_length = {c.history_length}")
print(f"coupling_strength = {c.coupling_strength}")

dt = 0.1
speed = 2.0
history_length = 10
coupling_strength = 1.0
```

Compatibility Layer

Here's how you make a model wrapper

- input: config file
- output: something you can `np.assert_allclose`

Since our test case format isn't directly compatible with either model, we build a translation layer to map our test case data into the input formats expected by TVB and Neurolib implementations.

```
In [5]: ## Example wrapper for tvb or neurolib
```

Delay testing

different connectivity configuration and different initial conditions

On the Pitfalls of Delay Testing

Be reaaaaaaaaally careful about your history shape

neviem ci tu je nieco ine? mozno parametre hodnoty 1 alebo 0?

```
In [8]: c.init_config_for_delays()
```

```
c.get_good_history_shape()
```

```
2025-11-05 11:34:46,788 - WARNING - tvb.basic.readers - File 'hemispheres' not found in ZIP.
```

```
init_hist_shape (46, 2, 76, 1)
```

```
Out[8]: (46, 2, 76, 1)
```

Pytest: Minimal and Extended Testing

Chceme tu mat tiez pytest, ze? Kvoli consistency

```
In [ ]: import numpy as np
import pytest

# TODO implement pytest
```

Running the Test Suite

TODO add an option for inputing test parameters using arguments in shell

Here you can run the `pytest` suite directly from this notebook.

Note: This executes tests as defined in the scripts and **will not reflect** changes made interactively in the notebook.

```
In [9]: # ! pytest test_case_file.py
```